Week 10 Monday

Lists: Collections That Grow

CS1

Essential Question

How do we keep track of many things at once, and how do we critique the code that does it?

Deitel & Deitel, Intro to Python for Computer Science and Data Science, Chapter 5, "Sequences: Lists and Tuples"; AI Fluency Lens 10 (Critique) tie-in

The Problem: One Variable Per Thing

```
colonist1 = "Ana"
colonist2 = "Bo"
colonist3 = "Cy"
print(colonist1)
print(colonist2)
print(colonist3)
```

- How would you print 50 names? Count them? Add one more later?
- We need *one name* for *many values*.

A List Holds Many Values in Order

```
colonists = []                    # empty list
colonists.append("Ana")           # add one at a time
colonists.append("Bo")
colonists.append("Cy")
print(colonists)                  # ['Ana', 'Bo', 'Cy']
print(len(colonists))             # 3
```

- `[]` makes an empty list; `.append(x)` adds to the end.
- `len(list)` is the number of items.
- **This is your gate:** build a list with `.append()` at least twice, then loop over it.

Index: Position Starts at 0

```
colonists = ["Ana", "Bo", "Cy"]
colonists[0]     # 'Ana'
colonists[2]     # 'Cy'
colonists[-1]    # 'Cy'  (last item)
colonists[3]     # IndexError: list index out of range
```

- Counting starts at 0; the last valid index is `len(list) - 1`.
- Negative indexes count from the end.

Traverse: Do Something With Every Item

```
for name in colonists:
    print("Welcome,", name)

total = 0
for name in colonists:
    total += 1
print("Headcount:", total)        # same as len(colonists)
```

- `for item in list:` visits each item in order.
- A traversal should *report something real*: a roster, a count, a search result.

Changing a List

```
crew = ["Ana", "Bo", "Cy"]
crew.insert(1, "Di")      # ['Ana', 'Di', 'Bo', 'Cy']
crew.remove("Bo")         # ['Ana', 'Di', 'Cy']
last = crew.pop()         # last == 'Cy'; crew == ['Ana', 'Di']
"Ana" in crew             # True
crew.index("Di")          # 1
```

- `.insert(i, x)`, `.remove(x)`, `.pop()`, `in`, `.index(x)`.
- Lists are **mutable**: these methods change the list itself.

Sorting: Two Different Tools

```
nums = [3, 1, 2]
sorted(nums)        # [1, 2, 3]   new list; nums unchanged
nums                # [3, 1, 2]
nums.sort()         # changes nums itself
nums                # [1, 2, 3]
x = nums.sort()     # x is None !
```

- `sorted(list)` returns a new list. `list.sort()` changes the list and returns `None`.

Critique Mode: Find the Weakness

- Monday Moments this week: **Lens 10, Critique** — find weaknesses in work before trusting it.
- Reviewing code is the same habit: read it, predict what it does, then test.
- Next: three snippets that *look* fine and are not.

Hazard 1: A Second Name Is Not a Copy

```
crew = ["Ana", "Bo", "Cy"]
backup = crew
backup.append("Di")
print(crew)          # what prints?
```

Hazard 1: A Second Name Is Not a Copy

```
crew = ["Ana", "Bo", "Cy"]
backup = crew
backup.append("Di")
print(crew)          # what prints?

# ['Ana', 'Bo', 'Cy', 'Di']  -- crew changed too!
backup = crew.copy()       # or list(crew)
```

- `backup = crew` makes two names for *one* list.
- `.copy()` makes a new list.

Hazard 2: Don't Remove While You Loop

```
crew = ["Ana", "Bo", "Bo", "Cy"]
for name in crew:
    if name == "Bo":
        crew.remove(name)
print(crew)          # what prints?
```

Hazard 2: Don't Remove While You Loop

```
crew = ["Ana", "Bo", "Bo", "Cy"]
for name in crew:
    if name == "Bo":
        crew.remove(name)
print(crew)          # what prints?

# ['Ana', 'Bo', 'Cy']  -- one Bo survived!
crew = [n for n in crew if n != "Bo"]   # build a new list
```

- Removing shifts the items, so the loop skips one.
- Safer: loop over a copy, or build a new list.

Hazard 3: sort() Returns None

```
scores = [90, 72, 85]
ranked = scores.sort()
print(ranked)        # what prints?
```

Hazard 3: sort() Returns None

```
scores = [90, 72, 85]
ranked = scores.sort()
print(ranked)        # what prints?

# None
ranked = sorted(scores)      # or: scores.sort(); then use scor
```

- Methods that change a list in place usually return `None`.

Your Gate: The Census

- Your world's population, inventory, case load, or customers becomes a list.
- **Quick Check (pass/fail):**
  - The list holds several distinct entities from your world.
  - It is built with `.append()` at least twice (not one typed-out literal).
  - At least one loop or `len()` reports something real.
- World Bible: one line — what you passed, what broke.

Work Time: Start Your Census

```
colonists = []
colonists.append("...")     # at least two appends
colonists.append("...")
for c in colonists:
    print(c)
print("Headcount:", len(colonists))
```

- Work for about 10 minutes; ask a neighbor to critique before you ask me.

Wrap-Up and Preview

- Lists: `[]`, `.append`, `len`, index, `for`, `in`.
- Hazards: aliasing, removing while looping, `.sort()` returns `None`.
- **Week 11:** dictionaries — look up an item by name or ID instead of searching the whole list.
- Gate and optional bonus practice are due **Friday 11:59 PM**.